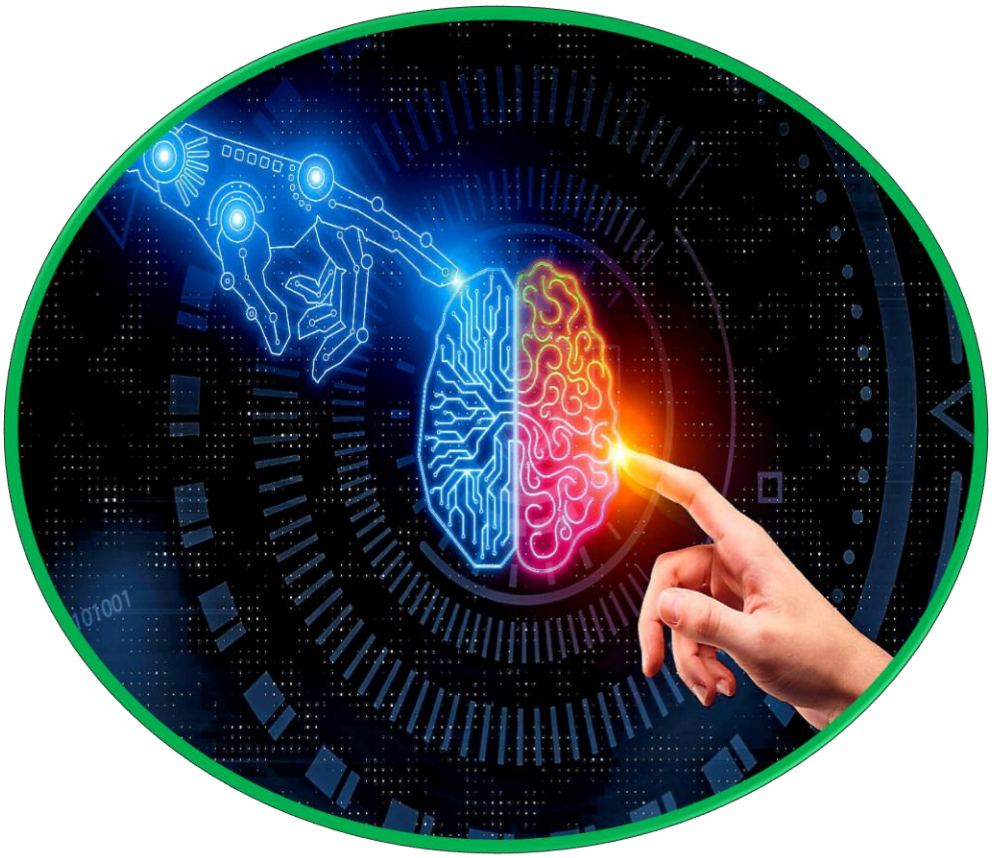
UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026

INFORME

MÉTODOS EN JAVA

PARÁMETROS, VOID Y RETORNO DE VALORES



Integrantes:

Esteban Zambrano

Kevin Garcés

Cristian Gómez

Erick Cordonez

Docente: Ing. José Ruben Caiza Caizabuano

1. Introducción

La programación orientada a objetos (POO) organiza el código modelando objetos del mundo real. Cada objeto tiene datos que lo describen (atributos) y comportamientos que puede realizar (métodos). Por ejemplo, una taza tiene color, capacidad y material, y puede llenarse, vaciarse o mostrar su contenido. En Java, esos comportamientos se programan mediante métodos.

El presente informe resume la exposición titulada «Métodos en Java: parámetros, void y retorno de valores». En él se describen, en el mismo orden de la presentación, los conceptos explicados, los ejemplos de código, los ejercicios resueltos con el grupo y las dos actividades de evaluación: la prueba diagnóstica aplicada al inicio de la clase y el Kahoot realizado al final.

La sesión tuvo una duración aproximada de dos horas: se inició con la prueba diagnóstica y un breve recordatorio del tema más fallado, se continuó con la explicación guiada (aproximadamente una hora y media) y se cerró con un Kahoot de 30 minutos sobre lo visto en clase.

2. Objetivos

2.1. Objetivo general

Explicar qué es un método en Java, cómo se estructura y cuándo se utiliza un método sin retorno (void) o un método con retorno de valores (return), mediante ejemplos, ejercicios y actividades de evaluación.

2.2. Objetivos específicos

- Definir qué es un método en POO y reconocer su importancia para modularizar el código y evitar su duplicación.
- Diferenciar los conceptos de clase y objeto, y ubicar los métodos como las acciones que el objeto puede realizar.
- Identificar las partes de la sintaxis de un método: modificador de acceso, tipo de retorno, nombre y parámetros.
- Distinguir cuándo usar un método void y cuándo usar un método con return.
- Resolver ejercicios prácticos de creación de métodos y verificar el aprendizaje con un cuestionario en Kahoot.

3. Desarrollo

3.1. Prueba diagnóstica y repaso inicial

Al inicio de la clase se aplicó una prueba diagnóstica con el fin de conocer los saberes previos de los estudiantes y detectar los temas que presentaban mayor dificultad. A partir de los resultados se identificó el tema con más errores y se realizó un breve recordatorio del mismo antes de comenzar con el contenido nuevo. Esto permitió que la explicación

partiera de bases claras y que los vacíos anteriores no afectaran la comprensión de los métodos.

3.2. Presentación del tema

Una vez realizado el repaso, se presentó el tema de la exposición: Métodos en Java: parámetros, void y retorno de valores. Se explicó que la clase se dividiría en tres partes: qué es un método y cuál es su estructura, los métodos sin retorno (void) y los métodos con retorno de valores (return), con ejercicios intercalados para practicar cada parte.

3.3. ¿Qué es un método en POO?

Un método es un bloque de código que realiza una tarea específica y se ejecuta solo cuando es llamado. Sus principales ventajas son:

- Modularidad: permiten dividir un programa grande en partes pequeñas y manejables.
- Reutilización: evitan la duplicación de código, porque una misma tarea se escribe una vez y se llama las veces que sea necesario.

En la presentación se utilizó un esquema que explica la POO a partir de un objeto concreto: una taza. Sus datos son la información que describe al objeto (color azul, capacidad de 350 ml, material cerámica) y sus comportamientos son las acciones que puede realizar (llenar(), vaciar() y mostrarContenido()). La POO, en resumen, organiza el código modelando objetos del mundo real.

Parte del objeto	Qué representa	Ejemplo (taza)
Datos	Información que describe al objeto	color: azul; capacidad: 350 ml; material: cerámica
Comportamientos	Acciones que el objeto puede realizar	llenar(), vaciar(), mostrarContenido()

3.4. Clase y objeto: ejemplos

Para ubicar a los métodos dentro de la POO se explicó la diferencia entre clase y objeto. La clase es el molde o plantilla que define las características y comportamientos que tendrán los objetos; define qué puede tener y qué puede hacer. El objeto es una instancia concreta de una clase: algo real creado a partir de ella, con valores reales en sus atributos, que puede usar los comportamientos definidos. En palabras de la presentación: la clase describe, el objeto existe.

Se emplearon los siguientes ejemplos:

- Clase automóvil: sus atributos o características son el color y las puertas, y sus métodos son las acciones que puede realizar: arrancar, frenar y acelerar (en el

esquema, acelerar(), frenar() y tocarBocina()). Un objeto de esa clase sería un auto de color rojo con 4 puertas.

- Clase Celular: con atributos como marca, modelo y color. Se planteó al grupo la pregunta «¿Qué funciones o métodos tiene el celular?», para que los estudiantes identificaran por sí mismos las acciones que puede realizar (por ejemplo, llamar o enviar mensajes).
- Analogía del molde de galletas: el molde representa la clase y la galleta ya horneada representa el objeto; con un mismo molde se pueden obtener muchas galletas.

Concepto	Qué es	Ejemplo
Clase	Molde o plantilla que define atributos y métodos	Automóvil: color, puertas, acelerar(), frenar(), tocarBocina()
Objeto	Instancia concreta creada a partir de la clase	Un auto rojo de 4 puertas
Atributo	Dato que describe al objeto	color: rojo; puertas: 4
Método	Acción que el objeto puede realizar	acelerar(), frenar(), tocarBocina()

3.5. Anatomía de un método en Java

Todo método en Java se compone de un modificador de acceso, un tipo de retorno, un nombre y una lista de parámetros entre paréntesis, seguidos de un cuerpo delimitado por llaves:

```
public tipoRetorno nombreDelMetodo(tipoParametro parametro1) {
    // Cuerpo del método: instrucciones a ejecutar
    // [return valor;] (si aplica)
}
```

Elemento	Descripción
public	Modificador de acceso.
tipoRetorno	Tipo de dato que el método devolverá al terminar (int, String, double, o void si no devuelve nada).
nombreDelMetodo	Identificador del método, por ejemplo calcularSuma.
(parámetros)	Datos de entrada que recibe el método. Pueden estar vacíos.

Como ejemplo de un método sin parámetros y sin retorno se mostró la firma:

public void mostrarMensaje() { ... }

3.6. Ejercicio 1: área de un cuadrado

Enunciado: crear un método que calcule el área de un cuadrado. Se sabe que recibe el valor de un lado (int lado) y debe devolver un número entero con el área calculada. ¿Cómo sería la estructura del método?

Respuesta correcta:

```
public int calcularAreaCuadrado(int lado) {
    int area = lado * lado;
    return area;
}
```

Explicación: se usa int antes del nombre porque el método va a retornar un valor numérico entero, y se usa return para entregar el resultado del cálculo. El parámetro int lado es el dato de entrada con el que se calcula el área.

3.7. Métodos sin retorno (void)

Los métodos void se utilizan cuando el método realiza una acción (como imprimir en consola, actualizar un estado o mostrar un menú) pero no necesita devolver ningún resultado de vuelta al código que lo llamó. Utilizan obligatoriamente la palabra reservada void en lugar del tipo de dato.

```
public class Cafeteria { // O cualquier clase de ejemplo
    // Método void que solo imprime un mensaje
    public void mostrarSaludo() {
        System.out.println("¡Bienvenido al sistema de gestión!");
    }
}
```

Cómo se genera (se invoca): a través de un objeto de la clase, con su nombre seguido de paréntesis.

```
objeto.mostrarSaludo();
```

3.8. Ejercicio 2: clase TurnoBanco

Enunciado: se requiere la clase TurnoBanco y se quiere un método que simplemente imprima en pantalla un mensaje de bienvenida con el turno asignado, sin necesidad de calcular ni devolver nada a otra variable. ¿Cómo lo escribirían usando void?

Respuesta:

```
public void mostrarTurno(int numeroTurno) {
    System.out.println("Su turno asignado es el: " + numeroTurno);
}
```

Explicación: lleva void porque la única tarea del método es ejecutar una acción visual (imprimir en consola), por lo que no requiere la instrucción return. Cabe notar que un método void sí puede recibir parámetros, como numeroTurno en este caso.

3.9. Métodos con retorno de valores (return)

Los métodos con retorno se utilizan cuando el método procesa información y necesita entregar un resultado para que otra parte del programa lo use. Las reglas clave son:

- El tipo de retorno en la firma debe coincidir exactamente con el tipo de dato que devuelve la instrucción return.
- La palabra reservada return finaliza la ejecución inmediata del método.

Ejemplo de código

```
public class Operacion {
    // Método que recibe dos enteros y retorna un entero con la suma
    public int sumar(int a, int b) {
        int resultado = a + b;
        return resultado; // Retorna el valor calculado
    }
}
```

Cómo se genera (se invoca y se captura el valor)

```
Operacion op = new Operacion();
int total = op.sumar(5, 3); // 'total' ahora vale 8
System.out.println("La suma es: " + total);
```

Se crea un objeto de la clase Operacion, se llama al método sumar con los argumentos 5 y 3, el método calcula la suma y devuelve 8, y ese valor se captura en la variable total, que luego se imprime: La suma es: 8.

3.10. Ejercicio 3: verificar la mayoría de edad

Enunciado: en una tienda online se crea un método para verificar si un cliente es mayor de edad. Recibe la edad (int edad) y debe devolver un valor de tipo booleano (true si es mayor, false si es menor). ¿Cómo estructurarían este método?

Respuesta:

```
public boolean esMayorDeEdad(int edad) {
    if (edad >= 18) {
        return true;
    } else {
        return false;
    }
}
```

Explicación: el tipo de retorno de la firma es boolean, por lo que obligatoriamente cada camino del método debe devolver un valor true o false utilizando return. Aquí el método tiene dos caminos (el if y el else) y ambos terminan con un return.

3.11. Comparación entre void y return

Característica	Método void	Método con return
¿Devuelve un valor?	No	Sí
Tipo de retorno en la firma	void	int, boolean, String, double, etc.
Uso típico	Imprimir, actualizar un estado, mostrar un menú	Calcular y entregar un resultado
Uso de return	No lo requiere	Obligatorio, del mismo tipo que el declarado
Cómo se invoca	objeto.mostrarSaludo();	int total = op.sumar(5, 3);
Ejemplos de la clase	mostrarSaludo(), mostrarTurno(int numeroTurno)	sumar(int a, int b), calcularAreaCuadrado(int lado), esMayorDeEdad(int edad)

3.12. Organización de la sesión

Momento	Duración	Actividad
Inicio	[minutos]	Prueba diagnóstica y breve recordatorio del tema más fallado.
Desarrollo	Aprox. 1 h 30 min	Explicación guiada de métodos, void y return, con tres ejercicios resueltos.

Cierre	30 min	Kahoot sobre el tema visto.

3.13. Cierre de la sesión: Kahoot

Al finalizar la exposición se realizó un Kahoot sobre el tema visto. Sus preguntas se centraron en identificar la sintaxis de un método y en diferenciar cuándo usar void y cuándo usar un tipo de dato con la sentencia return. La actividad permitió reforzar los contenidos, comprobar el nivel de comprensión del grupo y otorgar los primeros puntos extra.

4. Análisis de resultados

El Kahoot realizado al final de la clase, al ser un cuestionario más dinámico, permitió comprobar de forma más atractiva lo aprendido durante la sesión y logró que el tema se comprendiera de mejor manera.

El formato interactivo influyó de manera positiva en el grupo. Las preguntas con tiempo limitado, el ranking en tiempo real y la competencia entre compañeros mantuvieron la atención de los estudiantes y favorecieron la participación. Además, al existir una recompensa (los primeros puntos extra), los estudiantes se esforzaron más por responder correctamente, lo que los llevó a recordar con mayor cuidado la sintaxis de un método y la diferencia entre void y return.

De esta manera, la explicación guiada, los ejemplos de código, los tres ejercicios resueltos en clase y el Kahoot final contribuyeron a una mejor comprensión del tema.

5. Conclusiones

- Los métodos son bloques de código que realizan una tarea específica y se ejecutan solo cuando son llamados; permiten modularizar el programa y evitar la duplicación de código.
- La clase es el molde que define atributos y métodos, y el objeto es la instancia concreta que usa esos comportamientos.
- La estructura de un método (modificador, tipo de retorno, nombre y parámetros) es la base para declararlo e invocarlo correctamente.
- Un método void ejecuta una acción sin devolver valor; un método con return entrega un resultado cuyo tipo debe coincidir con el declarado en la firma.
- Los ejercicios del área del cuadrado, el turno del banco y la mayoría de edad mostraron cómo elegir entre void y return según lo que el método debe hacer.
- La prueba diagnóstica inicial permitió reforzar el tema más débil, y el Kahoot final sirvió para verificar de forma dinámica lo aprendido.

6. Recomendaciones

- Practicar creando métodos propios con y sin parámetros, y con y sin retorno, para afianzar la sintaxis.
- Probar los ejemplos y ejercicios en un entorno de desarrollo, y modificar los tipos y valores para observar los errores que aparecen.
- Repasar la diferencia entre clase y objeto antes de trabajar con métodos.
- Usar nombres descriptivos para los métodos, como calcularAreaCuadrado o esMayorDeEdad.

7. Bibliografía

Oracle. (s. f.-a). Defining methods. The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/javaOO/methods.html

Oracle. (s. f.-b). Returning a value from a method. The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/javaOO/returnvalue.html

Schildt, H. (2022). Java: A beginner's guide (9.ª ed.). McGraw-Hill.

W3Schools. (s. f.). Java methods. https://www.w3schools.com/java/java_methods.asp